COL362/632 Assignment 3: Out-of-Core Query Execution

Introduction

Most database servers have limited RAM to process data. When datasets grow or resources are limited, data systems rely on disk storage (of course for other reasons too). However, disk I/O is far slower than RAM. This makes smart utilization and minimal disk access critical for implementing efficient and effective data management systems.

In this assignment you will design a **query execution engine** that operates under a strict memory budget, optimizing it for minimal disk I/O and efficient in-memory computations.

System Architecture

```
┌──────────────────────────────────────────┐
│                 Monitor                  │
│  (spawns processes, sends queries, validates output)│
└──────────────────────────────────────────┘
             │ FD 5 (query in)      │ FD 6 (results / commands out)
             ▼                      ▲
┌──────────────────────────────────────────┐
│              Database (YOU)              │
│  reads query → accesses disk → outputs result rows  │
└──────────────────────────────────────────┘
             │ FD 4 (commands out)  │ FD 3 (block data in)
             ▼                      ▲
┌──────────────────────────────────────────┐
│              Disk Simulator              │
│  (block storage, tracks I/O for scoring) │
└──────────────────────────────────────────┘
```

To give you a hands-on experience with building data systems, we've designed an in-house system in Rust comprising three components:

1. **Disk Simulator** — simulates a block-addressable storage. It stores raw table files and provides a read/write anonymous space for your database to use as scratch. It tracks I/O (reads and writes) access patterns and estimates simulated disk time, which contributes to your score (more on this later).
2. **Monitor** — This is the orchestrator. It spawns the disk and database processes, sends the query, and validates your output against expected results.
3. **Database** — **this is what you will implement**. It receives a query from the monitor, reads table data from the disk, executes the query, and streams the result rows back to the monitor.

You typically need to touch only the `database/` directory.

File Descriptors

The monitor maps the following file descriptors for your database process at startup:

FD	Direction	Purpose
3	Read	Receive data from disk simulator
4	Write	Send commands to disk simulator
5	Read	Receive data from monitor (query JSON)
6	Write	Send data to monitor (results/commands)

These are already wired up in `database/src/io_setup.rs` — you do not need to open any files or sockets yourself.

Constraints

The monitor enforces the following hard limits on your database process:

1. **Memory**: At least 64 MB virtual address space (`RLIMIT_AS`), including the stack (`RLIMIT_STACK`). Note, you will not be able to load entire tables into memory as they typically will be of several gigabytes.
2. **Single-threaded**: No spawning subprocesses (`RLIMIT_NPROC = 1`).
3. **No file I/O**: Writing files on disk is forbidden (`RLIMIT_FSIZE = 0`). Use the disk simulator's anonymous space for any scratch storage.
4. **Time limit**: A generous time limit prevents infinite loops. Details will follow soon as assignment progresses.

Specifications

Query Specification

The monitor sends your database a single JSON line on FD 5 representing the query to execute. Starter code already parses this into a `Query` struct — see `common/src/query.rs`.

AST Node Types

A query is a tree of `QueryOp` nodes. The five node types are:

Node	Description
`Scan`	Leaf operator that reads all rows of a table from disk.
`Filter`	A unary operator that passes only rows satisfying all predicates.
`Project`	A unary operator that outputs a subset of columns, optionally renaming them. Preserves row order from its child.
`Sort`	A Unary operator that outputs rows sorted by a list of column ordering specification.
`Cross`	A binary operator that computes the Cartesian product of two child relations.

Operation Semantics

- **Project** preserves the row order of its child. If a `Project` appears as an ancestor of `Sort`, the final output will reflect that sort order even though sort happened before projection — this is intentional.
- **Filter**, **Cross** and **Scan** do **not** guarantee any particular output ordering. You are free to produce rows in any order for these operators.
- An SQL query can be represented as a combination of these five operators arranged in the correct tree structure.

Example

Consider:

```
SELECT a1 AS id, b2 FROM A JOIN B ON A.a1 = B.b1 WHERE b3 > 0 ORDER BY a2 ASC, b
2 DESC;
```

The AST is (innermost executed first):

```
Project(a1→id, b2→b2)
  └── Sort(a2 ASC, b2 DESC)
        └── Filter(b3 > 0)
              └── Filter(a1 = b1)
                    └── Cross
                          ├── Scan(A)
                          └── Scan(B)
```

Notice that `Sort` appears before `Project` in the tree — this is because `a2` would be dropped by
`Project` before `Sort` could use it. Since `Project` is order-preserving, the final output is still sorted correctly.

JSON Format

The `Query` struct serializes directly to/from JSON. The field names in JSON match the Rust struct fields. Here is the JSON for the example above:

```
{
  "root": {
    "Project": {
      "column_name_map": [["a1", "id"], ["b2", "b2"]],
      "underlying": {
        "Sort": {
          "sort_specs": [
            {"column_name": "a2", "ascending": true},
            {"column_name": "b2", "ascending": false}
          ],
          "underlying": {
            "Filter": {
              "predicates": [{"column_name": "b3", "operator": "GT", "value":
{"I32": 0}}],
              "underlying": {
                "Filter": {
                  "predicates": [{"column_name": "a1", "operator": "EQ", "valu
e": {"Column": "b1"}}],
                  "underlying": {
                    "Cross": {
                      "left":  {"Scan": {"table_id": "A"}},
                      "right": {"Scan": {"table_id": "B"}}
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Filter Predicates

A `Filter` node holds a list of predicates; a row passes only if **all** predicates are satisfied.

```
Predicate = { column_name, operator, value }

operator ∈ { EQ, NE, GT, GTE, LT, LTE }

value ∈ { Column(name) | I32(i32) | I64(i64) | F32(f32) | F64(f64) | String(stri
ng) }
```

When `value` is `Column(name)`, compare the row's value in `column_name` against its value in the named column. Otherwise compare against the literal.

Query Assumptions

1. The query is always valid (well-typed predicates, no missing columns, no ambiguous names).
2. The two children of a `Cross` node will never share a column name.
3. Strings contain only ASCII characters (a–z, A–Z, 0–9, and some special characters). String comparison is lexicographic by ASCII code point.
4. Column names contain no whitespaces.

Table Specification

Each table is described by a `TableSpec` in `db_config.json`:

```
{
  "name": "customer",
  "file_id": "customer",
  "column_specs": [
    {"column_name": "c_custkey", "data_type": "Int64", "stats": null},
    {"column_name": "c_name",    "data_type": "String", "stats": [...]},
    ...
  ]
}
```

- `name` is used to match tables referenced in `Scan` nodes.
- `file_id` is used to query the disk simulator for the file's location.
- `column_specs` lists columns in the order they appear in each row on disk.

Data Types

Type	Size	Encoding
`Int32`	4 bytes	Signed, little-endian
`Int64`	8 bytes	Signed, little-endian
`Float32`	4 bytes	IEEE-754 single precision, little-endian
`Float64`	8 bytes	IEEE-754 double precision, little-endian
`String`	variable	Null-terminated UTF-8 bytes

Statistics

Statistics may optionally be provided for each column. Your code must work correctly whether or not any statistics are provided; they are only hints for optimization.

> **Note:** Statistics are approximations — they resemble the data closely but are not exact.

Statistic	Meaning
`IsPhysicallyOrdered`	Values in this column are in non-decreasing order when rows are read sequentially from disk.
`RangeStat`	Inclusive `[lower_bound, upper_bound]` range of values in this column.
`HistogramStat`	A histogram of `(Range, Frequency)` pairs estimating value distribution.
`CardinalityStat`	Estimated total number of rows in the table. (This is technically a table statistic, if provided for multiple columns, the values will be close to each other).
`DensityStat`	Ratio of unique values to total rows (i.e. `num_distinct / num_rows`).

Disk Simulator Specification

Block Model

The disk is a flat array of fixed-size blocks. The block size is at least 512 bytes (typically 4096) and is the same for all blocks. Block IDs start at 0. A block is the smallest addressable unit of disk.

The disk is divided into two regions:

- **Read-Only (RO) Region** — blocks `0` to `anon-start-block - 1`. Contains files. Write attempts here will fail. Reads to blocks not belonging to any file will fail.
- **Read-Write (RW) Anonymous Region** — blocks `anon-start-block` to `2^64 - 1`. Use this as scratch space (intermediate results, sorting buffers, etc.). A block must be written before it can be read.

> **Important:** Please read the FAQ on anonymous region usage.

A file on disk is a continuous sequences of blocks. A file starts at its `start-block` block ID and spans its `num-blocks` number of blocks.

> **Important:** Files on disk need not be placed sequentially one after another, query for `start-block` and `num-blocks` of a file to know where exactly it is and how many blocks it spans.

Commands

All commands and responses end with `\n` unless noted. Always flush after writing commands.

Get commands (database → disk):

```
get block-size
```

Returns the block size in bytes as a decimal integer.

```
get block <start_block_id> <num_blocks>
```

Returns exactly `num_blocks * block_size` raw bytes, which correspond to `num_blocks` blocks starting from `start_block_id`. **The response does not end with `\n`.**

```
get file start-block <file_id>
```

Returns the starting block ID of the file as a decimal integer.

```
get file num-blocks <file_id>
```

Returns the number of blocks the file spans as a decimal integer.

```
get anon-start-block
```

Returns the block ID at which the anonymous (writable) region starts.

Put commands (database → disk):

```
put block <start_block_id> <num_blocks>
```

(ends with `\n`) Followed by exactly `num_blocks * block_size` raw bytes (not ending with `\n`). Writes the `num_blocks` blocks of data starting at `start_block_id`.
All the blocks must be in the anonymous region (`>= anon-start-block`). No response is sent.

> 💡 Example to put 2 blocks of data at block ID 42 and 43, you would send
> `put block 42 2\n<2*block_size number of bytes to put starting from block 42>`
> to disk

> 💡 Note: Remember to flush your output.

Exit (optional):

```
exit
```

Terminates the disk simulator cleanly.

Raw File Format

Each `.bin` file represents one table. The file size is always an exact multiple of `block_size`.

Block Layout

```
byte 0
┌─────────────────────────────────────────────────┐
│  <Row 1 data><Row 2 data>.......................│
│  ...............................................│
│  ...............................................│
│  ...............................................│
│  ...............................................│
│  ...............................................│
│  ...............................................│
│  ...............................................│
│  ...............................................│
│  ...............<Row N-1 data><Row N data>......│
│  ...............................................│
│  ...............................................│
│  .....(unused padding bytes)....................│
│  ...............................................│
│  ...............................................│
│  ..........<row_count[2 bytes, u16, little-endian]>│
└─────────────────────────────────────────────────┘
                                          byte block_size−1
```

- The last 2 bytes of every block hold `row_count` as a **unsigned 16-bit little-endian integer**.
- Rows are packed starting at byte 0 with no padding between them.
- Usable data space per block = `block_size - 2` bytes.
- A row is always fully contained within a single block — it never spans two blocks.

Row Encoding

Columns are stored sequentially in the order defined by `column_specs`. Each value is encoded as:

Type	Encoding
`Int32`	4 bytes, signed little-endian
`Int64`	8 bytes, signed little-endian
`Float32`	4 bytes, IEEE-754 LE
`Float64`	8 bytes, IEEE-754 LE
`String`	UTF-8 bytes followed by a `0x00` null terminator

Example

Schema: `(id: Int32, name: String)`
Rows: `(0xDEADBEEF, "hi")` and `(0x1BADB002, "DBMS")`

Encoded bytes (hex):

```
EF BE AD DE  68 69 00  02 B0 AD 1B  44 42 4D 53 00
^── id1 ──^  ^name1^  ^─── id2 ──^  ^── name2 ───^
```

Monitor Interface

Receiving the Query

On startup, read one line from Monitor. It is a JSON-encoded `Query`. Parse it with `serde_json::from_str`. (starter code does it for you).

Querying the Memory Limit

At any point before producing output, you may ask the monitor for the memory limit:

1. Write `get_memory_limit\n` to Monitor.
2. Read one line from Monitor. It is the memory limit in MB as a decimal `u64`.

Sending Results

Once query execution is complete, stream results to Monitor as follows:

1. Send `validate\n` — signals the start of output.
2. For each row in the query result (in the correct order), send one line:

   ```
   <col1>|<col2>|...|<colN>|
   ```

   Values are formatted as plain text (integers and floats as decimals, strings as-is). Each row ends with a trailing `|` and a `\n`.
3. After all rows, send `!\n` — signals end of output.
4. Remember to flush the output.
5. At this point your database program can exit cleanly. (i.e. you need to execute only a single query).

Example — a row `(26, "Alice", "9487882379")` is output as:

```
26|Alice|9487882379|
```

> **Note:** The monitor guarantees that no result rows contains a `|` character in any string column, so this delimiter is unambiguous.

Floating Point Output

When outputting `Float32` or `Float64` values, format them to match the output that SQLite would
produce for the same value. *(Details to be announced.)*

Getting Started

Step-0: Prerequisites and Build

- **Platform**: Linux, macOS, or Windows with WSL.
- **Dependencies**: Rust toolchain, `sqlite3` command-line tool.
- **Start code**: clone the repository my-awesome-db.

```
# From the repository root
cargo build -r          # builds all binaries in release mode
```

Compiled binaries appear in `target/release/`.

Step-1 Importing data into the Disk Simulator

Since our database doesn't support `INSERT` Queries we have provided you with a program that adds the data into the disk simulator directly.

We will now show you how to add the provided dataset (TPCH) into the disk simulator. Later we also show you how to add custom datasets.

1. Download the `tpch_scratch.tar.gz` starter from datasets.
2. Run `tar -xf tpch_scratch.tar.gz` **in the root directory**.
3. This will create a new directory called `scratch` as follows:-

```
scratch/
├── datasets/                # Source CSV + schema files
│   └── tpch/
│       ├── customer.csv
│       ├── customer.schema
│       ├── orders.csv
│       ├── orders.schema
│       └── ...
├── compiled_datasets/       # Empty (Will be filled later)
│   └── tpch/
└── runtimes/                # Empty (Will be filled later)
    └── tpch/
```

1. Now we will generate the binary files that can be read by the disk simulator along with some configuration.

```
cargo run -r --bin generator -- all \
    -d scratch/datasets/tpch \
    -c scratch/compiled_datasets/tpch \
    -r scratch/runtimes/tpch \
    -b target/release \
    -s 4096
```

Flag	Meaning
`-d`	Dataset folder (CSV + schema files)
`-c`	Compiled dataset folder (output: `.bin` files, `sqlite.db`)
`-r`	Runtime folder (output: all config JSON files)
`-b`	Build path — directory containing the compiled `disk` and `database` executables
`-s`	Block size in bytes (optional, default `4096`)

1. After running the above command you should see some new files in `compiled_datasets` and `runtimes` directory.

```
scratch/
├── datasets/                        # Source CSV + schema files
│   └── tpch/
│       ├── customer.csv
│       ├── customer.schema
│       ├── orders.csv
│       ├── orders.schema
│       └── ...
├── compiled_datasets/               # Generated binary files and SQLite DB
│   └── tpch/
│       ├── customer.bin
│       ├── orders.bin
│       └── sqlite.db                # Equivalent sqlite database which can be
│                                      used to verify the output.
└── runtimes/                        # Runtime configuration files
    └── tpch/
        ├── disk_sim_config.json     # generated
        ├── db_config.json           # generated
        ├── monitor_config.json      # generated (edit to add your queries)
        ├── expected_1.csv           # generated (empty placeholder, fill in)
```

Creating Custom Dataset

You may want to create a custom dataset for debugging purposes. For this you can create your own folder inside datasets folder just like `tpch` and add your dataset tables as follows:

Each table needs two files sharing the same base name:

CSV file (`.csv`) — one row per line, columns separated by `|`, with a trailing `|`:

```
1|Alice|Engineer|
2|Bob|Manager|
```

Schema file (`.schema`) — two lines:

```
id|name|job|
I32|String|String|
```

Line 1: column names separated by `|`.
Line 2: data types separated by `|`. Supported types: `I32`, `I64`, `Float32`, `Float64`, `String`.

Place both files in `scratch/datasets/mydataset/` and run.

```
cargo run -r --bin generator -- all \
    -d scratch/datasets/mydataset \
    -c scratch/compiled_datasets/mydataset \
    -r scratch/runtimes/mydataset \
    -b target/release \
    -s 4096
```

> Missing directories like `scratch/compiled_datasets/mydataset` and `scratch/runtimes/mydataset` will be automatically created by the generator if they don't exist.

Step-2: Writing Queries

The database you implement doesn't parse SQL — it consumes the query AST directly as JSON. Here is
how to generate that JSON.

1. Open `demo_query_printer/src/main.rs` and write your query using the builder API.
2. Run: `cargo run -r --bin demo_query_printer`
3. Copy the printed JSON into the `"query"` field of the relevant entry in `monitor_config.json`.

Editing `monitor_config.json`

The generator pre-fills absolute paths to the executables and config files. You only need to edit the `"query"`, `"execution_name"`, and `"disabled"` fields, and point `"expected_output_file"` at
your expected output (generated in the next step).

```
{
  "disk_config": {
    "disk_prog":        "/abs/path/target/release/disk",
    "disk_prog_config": "/abs/path/runtimes/tpch/disk_sim_config.json"
  },
  "database_config": {
    "database_prog":        "/abs/path/target/release/database",
    "database_prog_config": "/abs/path/runtimes/tpch/db_config.json"
  },
  "query_configs": [
    {
      "execution_name":       "simple_scan",
      "disabled":             false,
      "memory_limit_mb":      64,
      "query":                { "root": { "Scan": { "table_id": "customer" } }
},
      "expected_output_file": "/abs/path/runtimes/tpch/expected_1.csv"
    }
  ]
}
```

More query objects can be added to the `"query_configs"` array. Set `"disabled": true` to skip a
query without removing it.

Step-3: Generating Expected Output with SQLite

The expected output files are needed by the monitor for validation. Generate them using the SQLite
database created in Step-1.

1. Write your query in a `.sql` file, e.g. `my_query.sql`.
2. Run:

```
sqlite3 scratch/compiled_datasets/tpch/sqlite.db < my_query.sql > scratch/runtim
es/tpch/expected_1.csv
```

Important — trailing pipe trick: The monitor expects each row to end with `|`. SQLite does not add a trailing delimiter by default. Work around this by appending an empty string column to every
query:

Instead of:

```
SELECT a, b, c FROM table1;
```

Write:

```
SELECT a, b, c, '' FROM table1;
```

The empty `''` causes SQLite to output a trailing `|` for every row, matching the required format.

> Update `"expected_output_file"` in `monitor_config.json` to point to this file.

Step-4: Running the Query

```
cargo run -r --bin monitor -- --config <path/to/monitor_config.json>
```

Points to note regarding the monitor program:-
1. It only runs the enabled queries.
2. It runs the queries in sequence and prints success or validation errors.
3. It stops after encountering the first error.

> 💡 Rebuild the database when modifications are made, Monitor doesn't build for you.
>
> ```
> cargo build -r # or cargo build -r --bin database
> ```

Different Configuration Files

We already saw `monitor_config.json` above (in Step-2). Here are the other two config files for reference.

`disk_sim_config.json`

Generated automatically by `generator disk`. Describes the block size and paths to `.bin` files.

```
{
  "disk_config": {"block_size": 4096},
  "files": [
    {"id": "customer", "file_path": "/absolute/path/to/customer.bin"},
    {"id": "orders",   "file_path": "/absolute/path/to/orders.bin"}
  ]
}
```

`db_config.json`

Generated automatically by `generator database`.

```
{
  "table_specs": [
    {
      "name": "customer",
      "file_id": "customer",
      "column_specs": [
        {"column_name": "c_custkey", "data_type": "Int64",  "stats": null},
        {"column_name": "c_name",    "data_type": "String", "stats": null}
      ]
    }
  ]
}
```

> Statistics are also generated but not shown here!

Statistics Reference

Statistics are stored per-column in `db_config.json` as a list under `stats`. You may use them to make better decisions in your query execution strategy. Your code must function correctly even when `stats` is `null` or only a subset of statistics are provided.

Statistic	JSON key	Type
`IsPhysicallyOrdered`	`"IsPhysicallyOrdered"`	unit
`RangeStat`	`"RangeStat"`	`{lower_bound, upper_bound}` where bounds are typed values
`HistogramStat`	`"HistogramStat"`	`{frequency_points: [(Range, Frequency)]}`
`CardinalityStat`	`"CardinalityStat"`	`u64` row count estimate
`DensityStat`	`"DensityStat"`	`f32` in `[0, 1]`: fraction of rows with a unique value

Evaluation and Grading

Your score combines **simulated disk I/O time** (tracked by the disk simulator based on access patterns like sequential vs random access, transfer rates, seek behavior ..etc) and **code execution time** (CPU time of your process alone).

Grading uses a **tiered system** based on relative performance on a class leaderboard.

Tier	Requirement
Bronze	Logically correct output, terminates within time limit
Silver	Relative performance ranking
Gold	Relative performance ranking
Platinum	Relative performance ranking
Diamond	Relative performance ranking

You may resubmit multiple times and optimize your solution during the final weeks of the assignment to climb the leaderboard. More details will be announced as the assignment progresses.

Your solution will be scored based on the simulated IO time and CPU time (lower is better). Among correct submissions, the scores will be sorted by ascending score, and assigned tiers as follows:

- Diamond: Top 10% (lowest scores)
- Platinum: next 20%
- Gold: next 25%
- Silver: next 25%
- Bronze: bottom 20% (highest scores among correct ones)

The scores will then be normalized as 100 x (S_max - S)/(S_max - S_min)

Each tier also has multiplier:

- Diamond: x1.20
- Platinum: x1.15
- Gold: x1.10
- Silver: x1.05
- Bronze: x1.00

The final marks in the assignment will be calculated as:

min(100,Normalized Score x Multiplier) x 0.12

> **There is no report submission for this assignment.**

Profiling with Valgrind

Optimizing execution time in this assignment would be tricky due to involvement of interaction with disk simulator. A typical wall clock based time analysis might not yield good results.

To find CPU hotspots in your code, you can use tools like Callgrind from Valgrind and KCacheGrind.

They help you visualize hotspots, providing you a graphical view of hotspots and function calls.

You need to build your code in [optimized + debug] mode for best analysis, you can use the following command to build in that mode.

```
# Build with optimizations and debug info
cargo build --profile profiling   # defined in Cargo.toml
```

Setup of Valgrind is not shown here, we suggest you read this yourself and modify the monitor source code to incorporate this. (You may also need to remove some security options added in monitor code for this to work).

> This is not required but is helpful for identifying slow code paths.

FAQ

1. **Can we modify the starter code? Can we add Cargo packages?**

   You are free to modify any part of the starter code including the monitor, disk, and generator
   for local testing purposes or addition of additional traits. The final submission must produce a `database` executable that communicates via the protocols described in this document. The starter code already includes minimal packages sufficient for the assignment. If you believe an additional crate serves the learning objectives of the assignment, post on Piazza.

   Takeaway: The database executable comes from your code (monitor + disk + database + configs + fdwrapper ... all source codes combined). But the monitor and disk executable from our own source code, so all we require is that your database executable follows the database config provided, and runs using the exact protocols specified in this document.

2. **Can we use multiple threads?**

   No. The monitor sets `RLIMIT_NPROC = 1`, which prevents spawning additional processes or threads.

3. **Can we write temporary files to disk?**

   No. The monitor sets `RLIMIT_FSIZE = 0`. All scratch storage must go through the disk simulator's
   anonymous block region (blocks at or above `anon-start-block`).

4. **How do I handle floating point rounding?**

   (Details to be announced.)

5. **The monitor says validation failed — how do I debug?**

   Run your query against SQLite and compare the output line by line. Make sure you are applying the trailing pipe trick and that your row order matches (if the query has no `ORDER BY`, any order is accepted — but make sure your expected output file also has the same order). Use `disabled: true` to isolate a single query config.

6. **What is the size of anonymous space? Disk crashed by OOM? How is 2^64 blocks of disk space even possible?**

   The disk simulator currently stores the Read-Write Anonymous (scratch) blocks entirely in your system's RAM. To prevent your database from crashing the simulator by running out of memory, you must understand how the simulator allocates space:

   - **Virtual Address Space is Massive:** You are completely free to use *any* block ID between `anon-start-block` and `2^64 - 1`.
   - **Physical Allocation is Lazy (Sparse):** The simulator does *not* allocate memory for the empty space between block IDs. It only allocates `block_size` bytes of actual RAM when you write to a **new, unique** block ID.

   Example:
   Assume `anon-start-block` is 100, and your `block_size` is 4KB.

   - **Safe (Sparse Allocation):** Your database writes data to block `100` and then writes data to block `10,000,000`. Because you only touched 2 unique block id's, the simulator allocates exactly **8KB** of RAM. It ignores the millions of empty blocks in between.
   - **Unsafe (Exhausting Memory):** Your database writes data to every single block sequentially from `100` to `1,000,100`. You have now touched 1,000,000 unique blocks. The simulator will allocate **~4 Gigabytes** of RAM.

   The Takeaway: There is no restriction on *how high* your block IDs go, but there is a strict limit on the *total number* of unique block IDs your program touches. Design your database to reuse and recycle scratch blocks (e.g., via a Buffer Pool) to keep your memory footprint low. We will announce the hard limit on the total allowed anonymous size soon. (it will be at least 10GB per say or even higher depending on the size of datasets we decide to use.)